

Прокси и ограничение частоты запросов

Table of contents

Прокси и ограничение частоты запросов	1
Прямой и обратный прокси	1
Алгоритмы распределения нагрузки	2
Ограничение частоты запросов	5

Прокси и ограничение частоты запросов

Как только за приложением начинает стоять не один процесс, появляются два вопроса: куда направлять входящий запрос и как не дать кому-нибудь положить сервис. Пока сервер один, клиенты ходят напрямую. Запустили несколько копий ради масштабирования или резервирования — нужен кто-то впереди, кто решает, на какой инстанс отправить очередной запрос. С этим работают прокси-серверы, а правила распределения задают алгоритмы балансировки.

Ограничение частоты запросов закрывает вторую сторону: от перегрузок. Бот с тысячей запросов в секунду, внезапный всплеск трафика, пользователь со случайным бесконечным циклом — всё это останавливается на уровне прокси, не доходя до приложения.

Дальше разберём, как nginx, HAProxy и Envoy работают шлюзом перед сервисом и какие алгоритмы стоят за их настройками.

Прямой и обратный прокси

Прямой прокси (Forward Proxy):

Клиент → прямой прокси → Интернет

Прокси выступает от имени клиентов и скрывает их от внешних серверов.

Сценарии использования: - Корпоративные сети: контроль доступа сотрудников к интернету - VPN и обход блокировок (клиент подключается через прокси в другой стране) - Кеширование контента (прокси сохраняет часто запрашиваемые ресурсы) - Анонимизация (сервер видит IP прокси, а не клиента)

Пример:

Сотрудник компании → Корпоративный прокси → youtube.com

↓

Логирует запросы

Блокирует запрещенные сайты

Кеширует статику

Обратный прокси (Reverse Proxy):

Клиенты → обратный прокси → серверы приложения

Здесь прокси работает от имени серверов и прячет внутреннюю инфраструктуру от клиентов.

Сценарии использования: - Load balancing (распределение запросов по серверам) - SSL termination (расшифровка HTTPS на прокси, внутри — HTTP) - Кеширование статики и API-ответов - Защита от DDoS (ограничение частоты запросов, фильтрация) - Единая точка входа для микросервисов

Пример: nginx как обратный прокси

```
upstream backend {
    server app1.internal:3000;
    server app2.internal:3000;
    server app3.internal:3000;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
    }
}
```

Популярные решения: - **nginx** — лёгкий и быстрый, до 10K req/s на одном инстансе - **HAProxy** — специализированный балансировщик, TCP+HTTP - **Envoy** — современный cloud-native proxy (используется в Istio, service mesh) - **Traefik** — автоматическая конфигурация из Docker/Kubernetes labels

Алгоритмы распределения нагрузки

1. Round Robin (по кругу)

Запросы раздаются по серверам по очереди.

Пример:

```
Серверы: S1, S2, S3
Запросы: R1→S1, R2→S2, R3→S3, R4→S1, R5→S2, R6→S3...
```

Преимущества: - Простота реализации - Равномерное распределение запросов

Недостатки: - Не учитывает загрузку серверов (если S1 обрабатывает тяжелые запросы, он перегружен) - Не учитывает мощность серверов (S1 может быть слабее S2)

Когда использовать: все серверы одинаковые, запросы примерно одинаковой сложности.

2. Weighted Round Robin (взвешенный)

То же самое, но серверам выдаются веса: чем мощнее, тем больше запросов прилетает.

Пример:

```
Серверы: S1 (вес 1), S2 (вес 2), S3 (вес 1)
Запросы: R1→S2, R2→S2, R3→S1, R4→S3, R5→S2, R6→S2, R7→S1...
          ↑ S2 получает в 2 раза больше запросов
```

Когда использовать: серверы разной мощности (например, 2 мощных + 1 слабый для резервирования).

3. Least Connections (минимум соединений)

Новый запрос уходит на сервер, у которого сейчас меньше всего активных соединений.

Пример:

```
Состояние:
S1: 5 активных соединений
S2: 3 активных соединения
S3: 7 активных соединений

Новый запрос → S2 (минимум соединений)
```

Преимущества: - Учитывает реальную загрузку серверов - Подходит для долгоживущих соединений (WebSocket, gRPC streaming)

Недостатки: - Прокси должен отслеживать состояние соединений - Чуть сложнее, чем Round Robin

Когда использовать: запросы разной длительности, WebSocket, потоковые соединения.

4. IP Hash (хеширование по IP)

IP клиента хешируется, и все его запросы идут на один и тот же сервер.

Пример:

```
hash(IP клиента) % количество_серверов = номер сервера

Клиент 192.168.1.10 → hash → 2 → всегда S2
Клиент 192.168.1.20 → hash → 1 → всегда S1
```

Преимущества: - Привязка сессии к одному серверу - Не нужно внешнее хранилище сессий

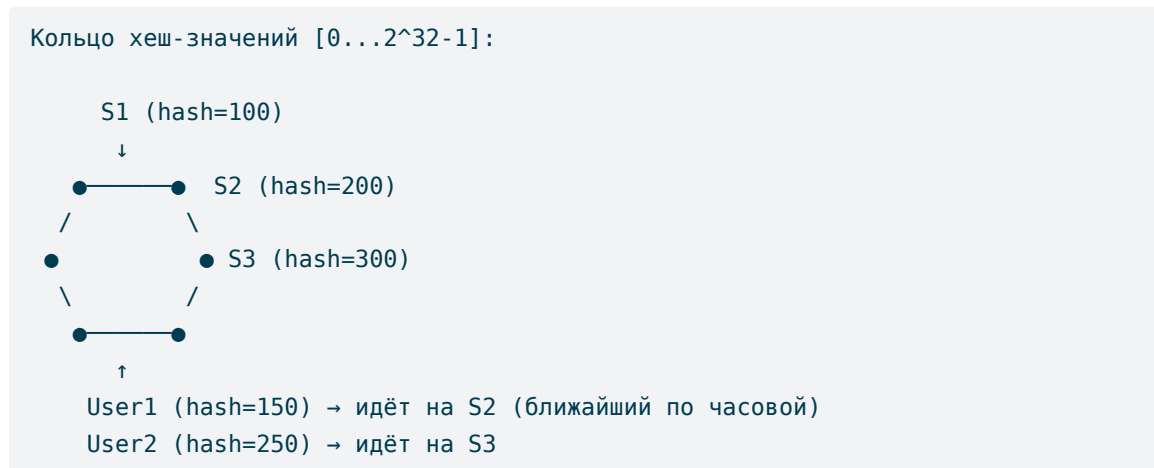
Недостатки: - Неравномерное распределение, если мало уникальных IP - При добавлении/удалении сервера — перераспределение клиентов

Когда использовать: нужна привязка сессии к серверу, нет централизованного хранилища сессий.

5. Consistent Hashing (консистентное хеширование)

Серверы и ключи (например, user ID) раскладываются на виртуальном кольце, а ключ уходит на ближайший сервер по часовой стрелке.

Пример:



Преимущества: - Минимальное перераспределение при добавлении/удалении сервера
 - Только $\sim 1/N$ ключей перераспределяются (N — число серверов) - Используется в распределенных кешах (Redis Cluster, Memcached)

Недостатки: - Сложнее реализация - Может быть неравномерное распределение (решается виртуальными узлами)

Когда использовать: - Распределенные кешы (Redis, Memcached) - Sharding баз данных - CDN (маршрутизация к ближайшему серверу)

Сравнение алгоритмов балансировки:

Алгоритм	Состояние	Привязка сессии	Масштабируемо	Когда использовать
Round Robin	Нет	Нет	Отлично (простое добавление серверов)	API без состояния, одинаковые серверы

Алгоритм	Состояние	Привязка сессии	Масштабируемо	Когда использовать
Weighted RR	Нет	Нет	Отлично	Серверы разной мощности
Least Connections	Да (счётчики)	Нет	Хорошо	Долгоживущие соединения, WebSocket
IP Hash	Нет	Да	Среднее (перераспределение при масштабировании)	Привязка без внешнего хранилища
Consistent Hashing	Нет	Да	Отлично (минимум перераспределения)	Распределенные серверы, кеширование БД

Ограничение частоты запросов

Зачем нужно: - Защита от DDoS и злоупотреблений - Контроль использования API (бесплатный план: 100 req/час, платный: 10K req/час) - Предотвращение перегрузки приложения

1. Token Bucket (ведро токенов)

Идея: у клиента есть ведро на B токенов, в которое каждые $1/R$ секунд добавляется ещё один токен. Каждый запрос забирает один токен; если ведро пустое — запрос отклоняется с 429 Too Many Requests.

Пример: лимит 10 req/s, запас 20 токенов

Начальное состояние: 20 токенов (ведро полное)

$t=0.0s$: 15 запросов → токены: $20 - 15 = 5$

$t=0.5s$: пополнение 5 токенов ($10 \text{ req/s} \times 0.5s$) → токены: $5 + 5 = 10$

$t=1.0s$: 3 запроса → токены: $10 - 3 = 7$

$t=1.5s$: пополнение 5 токенов → токены: $7 + 5 = 12$

$t=2.0s$: 25 запросов → доступно только 12 → принято 12, отклонено 13 ❌

Преимущества: - Разрешает кратковременные всплески - Простая реализация

Недостатки: - Если клиент долго не ходил в API, у него копится полное ведро и потом он может выстрелить всем лимитом сразу

Реализация (псевдокод):

```

type TokenBucket struct {
    tokens    float64 // текущее число токенов
    capacity  int           // максимум токенов
    rate      float64 // токенов в секунду
    lastRefill time.Time
}

func (b *TokenBucket) Allow() bool {
    now := time.Now()
    elapsed := now.Sub(b.lastRefill).Seconds()

    // Пополняем токены
    b.tokens = min(b.capacity, b.tokens + elapsed * b.rate)
    b.lastRefill = now

    if b.tokens >= 1.0 {
        b.tokens -= 1.0
        return true // Разрешено
    }
    return false // Отклонено
}

```

2. Sliding Window Log (скользящее окно с логом)

Храним timestamp каждого запроса в пределах окна (скажем, за последний час). Когда приходит новый запрос, выкидываем устаревшие записи и смотрим, сколько осталось. Меньше лимита — пропускаем.

Пример: лимит 5 req/час

```

Текущее время: 14:30

Лог запросов: [12:50, 13:20, 13:45, 14:10, 14:25]
Новый запрос в 14:30:
- Удаляем 12:50, 13:20 (старше 13:30, то есть старше часа)
- Остаётся: [13:45, 14:10, 14:25]
- 3 < 5 → разрешаем, добавляем 14:30
- Лог: [13:45, 14:10, 14:25, 14:30]

```

Преимущества: - Точный лимит (ровно N запросов в окне)

Недостатки: - Требуется хранить все timestamps → $O(N)$ памяти - Дорого для высокого лимита (100K req/час)

3. Sliding Window Counter (скользящее окно со счетчиками)

Время режется на фиксированные окна (например, по минуте), и для текущего момента берётся взвешенная сумма счётчиков текущего и предыдущего окна.

Пример: лимит 10 req/мин

```
Окна по 1 минуте:  
14:29:00-14:30:00 (прошрое): 8 запросов  
14:30:00-14:31:00 (текущее): 4 запроса  
  
Запрос в 14:30:30 (50% в текущем окне):  
Оценка = 8 × (1 - 0.5) + 4 = 4 + 4 = 8 запросов  
8 < 10 → разрешаем
```

Преимущества: - Компактно: храним только 2 счетчика - Приблизительно точный лимит

Недостатки: - Небольшая погрешность на границах окон

Популярные реализации: - **Redis** — **INCR** + **EXPIRE** для счетчиков - **nginx** — **limit_req_zone** (leaky bucket) - **Kong / API Gateway** — встроенные плагины ограничения частоты запросов